IMU-Based Step Counting

Causal TCN Event Detection and Real-Time Deployment

Mio Zhu

Department of Electrical and Electronic Engineering
The University of Hong Kong

Roadmap

1. Problem
2. Data
3. Pipeline
4. Model
5. Training
6. Inference
7. Evaluation
8. Deployment

Goal and Constraints

Task

Count walking steps from IMU recordings with one interface for offline evaluation and real-time updates.

Inputs

- Required: time and 3-axis acceleration
- Optional: gyro and magnetometer
- Irregular raw sample rates handled by resampling

Outputs

- Offline: step count and timestamps
- Streaming: new steps and cumulative total
- Diagnostics for threshold and model state

Design challenge

Detect sparse step events while staying robust to phone orientation, gait speed, sensor placement, and chunk boundaries.

Dataset and Ground Truth

- Training data: OxWalk Dec 2022 IMU recordings
- 39 participants used in the final retrain
- Four modalities: hip/wrist at 100 Hz and 25 Hz
- Participant-level split prevents the same person appearing in train and test

Split	Participants	Windows
Train	23	649,794
Val	8	210,000
Test	8	230,164

Label construction

Binary step annotations are resampled to 50 Hz, then widened with a Gaussian label of $\sigma = 0.06$ s for event learning.

Preprocessing and Windowing

1. Parse timestamped accelerometer CSV files and convert timestamps to seconds.
2. Resample every recording to a common 50 Hz timeline with linear interpolation.
3. Build an orientation-aware feature vector:

$$x_t = [a_x(t), a_y(t), a_z(t), \|a(t)\|_2]$$

4. Slice into 4 s windows with 0.5 s stride, giving 200 samples per window.
5. Standardize features using train-set mean and standard deviation saved in the checkpoint.

Why include acceleration magnitude?

The raw axes preserve motion direction, while $\|a\|_2$ gives a stable cue when phone orientation changes.

Causal TCN StepNet Architecture

Backbone

- Input projection: 4 channels $\rightarrow$ 64 channels
- 4 causal residual convolution blocks
- Kernel size 5, dilations $1, 2, 4, 8$
- BatchNorm, GELU, dropout 0.15

Two heads

- Event head: per-sample step probability
- Count head: expected number of steps in the window
- Causal convolutions support streaming inference

Acc window $\rightarrow$ feature normalization $\rightarrow$ causal TCN $\rightarrow$ event logits + count estimate

Models Compared in the Report

Method	Strengths	Observed limitations
ACF baseline	No training, lightweight, interpretable cadence estimate	Needs quasi-periodic walking; weaker at starts/stops and irregular arm motion
LSTM StepNet	Learns temporal patterns and gait rhythm	Sequential recurrence is slower to train/infer; no final checkpoint retained
Causal TCN StepNet	Parallel training, fixed streaming context, event + count heads	Needs labeled data and threshold calibration

Final choice

TCN was selected for the accuracy target; ACF remains a strong low-cost fallback. Final TCN test count MAE: 0.484 steps/window.

ACF Baseline: Classical Signal Pipeline

Processing steps

1. Compute $a_{mag}(t)$ from tri-axial acceleration
2. Remove slow baseline drift and smooth residual motion
3. Select strongest autocorrelation peak in 1.2–2.8 Hz walking band
4. Accept local maxima above adaptive threshold and separated by ρP samples

When it works

Regular walking creates a stable autocorrelation peak, so period and peak-distance estimates are reliable.

Failure modes

Start/stop transitions, irregular arm swing, and non-walking periodic motion weaken stationarity.

Self-recorded ACF results: test1 84 $\rightarrow$ 88, test2 100 $\rightarrow$ 97; MAE 3.5 steps, MAPE 3.88%.

Training Objective and Hyperparameters

Loss

$$L = L_{BCE}(y_{event}, \hat{y}_{event}) + 0.15 L_{MSE}(y_{count}, \hat{y}_{count})$$

Positive event samples use weight 3.0 to counter sparse labels.

- Optimizer: AdamW
- Learning rate: 10^{-3}
- Weight decay: 10^{-4}
- Batch size: 256
- Early stopping patience: 6 epochs
- Best epoch: 4

Checkpoint contents

The saved model stores network weights, feature normalization statistics, sample rate, window size, split metadata, and seed.

Offline and Real-Time Inference

Offline path

- Validate time and acceleration shape
- Sort timestamps and remove duplicates
- Normalize start time to zero
- Resample to model rate and call the same update logic

Streaming path

- Keep a 4 s context buffer
- Run causal TCN on context + new chunk
- Detect local maxima above probability threshold 0.68
- Enforce minimum step interval of 0.33 s

Boundary handling

Previous probabilities and timestamps are retained, so peaks crossing chunk boundaries are counted exactly once.

Experimental Results

Neural model split

Metric	Value
Best validation loss	0.3732
Test loss	0.4562
Test event loss	0.3364
Test count loss	0.7992
Test count MAE	0.4840

Test event loss is lower than validation loss, suggesting the test split may contain easier motion sequences.

Self-recorded sessions

Method	MAE	MAPE	Training
TCN	1.5	1.69%	>30 min CPU; ∼8 min GPU
ACF	3.5	3.88%	none

Stress-test subset

Deterministic subset MAE is 15.6 steps; wrist 25 Hz and hip 100 Hz contribute most error.

Real-Time Demo and Takeaways

phyphox demo

- Polls Remote Access API every 0.05 s
- Auto-detects time and acceleration buffers
- Displays acceleration curves, detected steps, and cumulative count
- Web dashboard available through Flask and Chart.js

Key lessons

- Causality matters for live feedback
- ACF is useful for lightweight fallback
- Participant split exposes generalization gaps
- Threshold and minimum interval convert probabilities into stable events

Limitations and next steps

TCN allows up to ∼180 steps/min with a 0.33 s interval, while ACF caps cadence at ∼168 steps/min. Future work: adaptive thresholds, placement adaptation, and hybrid ACF+TCN fallback.